

***BRACT's***

**Vishwakarma Institute of Technology, Pune**

**Practical Implementation Sheet**

Department: CSE-AI      Semester: 5      Academic Year: 2026-27

Division: E      Batch: 2      Practical No: 10

Roll no: 49      PRN No: 12414235      Name of Student: Saloni Tiwari

**Subject Name: Deep Learning**

**Problem Statement:**

Design and implement a deep learning application for audio classification that incorporates Speech-to-Text (STT) and Text-to-Speech (TTS) using an appropriate deep learning architecture, and evaluate its performance using suitable metrics.

**Algorithm:**

- **Import Libraries:** Import the required libraries for audio processing, deep learning, evaluation, speech recognition, and text-to-speech.
- **Load Dataset:** Upload and extract the RAVDESS dataset and collect speech audio files with their corresponding emotion labels.
- **Select Samples:** Select a balanced subset of audio samples for the eight emotion classes to reduce training time.
- **Audio Preprocessing:** Resample each audio file to 22,050 Hz, standardize its duration to 3 seconds, and extract a 64-bin Mel spectrogram.
- **Prepare Dataset:** Encode the emotion labels, reshape the features for CNN input, and split the dataset into training and testing sets.
- **Build CNN Model:** Construct a CNN using convolutional layers, pooling, global average pooling, dropout, and a softmax output layer.
- **Train Model:** Train the CNN on the training dataset and validate it using the validation split.
- **Evaluate Model:** Calculate test accuracy, generate a classification report, display a confusion matrix, and plot training and validation accuracy.
- **Save Model:** Save the trained CNN model for future predictions.
- **Load Whisper:** Initialize the Whisper speech recognition model.
- **Speech-to-Text:** Upload a WAV audio file and transcribe its spoken content using Whisper.

- **Emotion Prediction:** Preprocess the uploaded audio using the same feature extraction procedure and predict its emotion using the trained CNN.
- **Text-to-Speech:** Convert the recognized text into speech using gTTS and play the generated audio.
- **Display Results:** Display the transcription, predicted emotion, prediction confidence, evaluation results, and generated speech

## Code:

```
# SPEECH EMOTION RECOGNITION USING CNN, WHISPER AND gTTS

# 1. INSTALL AND IMPORT LIBRARIES
!pip -q install librosa soundfile scikit-learn matplotlib seaborn transformers gTTS

import os
import glob
import zipfile
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import librosa
import tensorflow as tf
import torch

from google.colab import files
from IPython.display import Audio, display
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import (
    Conv2D, MaxPooling2D, GlobalAveragePooling2D,
    Dense, Dropout
)
from tensorflow.keras.utils import to_categorical
from transformers import pipeline
from gtts import gTTS

# Allow TensorFlow to allocate GPU memory gradually when possible
for gpu in tf.config.list_physical_devices("GPU"):
    try:
        tf.config.experimental.set_memory_growth(gpu, True)
    except RuntimeError:
        pass

np.random.seed(42)
tf.random.set_seed(42)

# 2. UPLOAD AND EXTRACT RAVDESS DATASET
print("Upload the RAVDESS dataset ZIP file.")
uploaded_zip = files.upload()

zip_files = [
    name for name in uploaded_zip
    if name.lower().endswith(".zip")
]

if not zip_files:
    raise ValueError("Please upload the RAVDESS dataset as a ZIP file.")

dataset_path = "/content/RAVDESS"
os.makedirs(dataset_path, exist_ok=True)

with zipfile.ZipFile(zip_files[0], "r") as zip_ref:
    zip_ref.extractall(dataset_path)
```

```

print("Dataset extracted successfully.")

# 3. SELECT AUDIO FILES AND EXTRACT EMOTION LABELS
emotion_map = {
    "01": "neutral",
    "02": "calm",
    "03": "happy",
    "04": "sad",
    "05": "angry",
    "06": "fearful",
    "07": "disgust",
    "08": "surprised"
}

all_files = glob.glob(
    os.path.join(dataset_path, "**", "*.wav"),
    recursive=True
)

samples = {emotion: [] for emotion in emotion_map.values()}

for file_path in all_files:
    filename = os.path.basename(file_path)
    parts = filename.replace(".wav", "").split("-")

    # RAVDESS filename: modality-channel-emotion-...
    if len(parts) >= 7 and parts[1] == "01":
        emotion_code = parts[2]

        if emotion_code in emotion_map:
            emotion = emotion_map[emotion_code]

            # Limit samples per class to reduce training time
            if len(samples[emotion]) < 60:
                samples[emotion].append(file_path)

audio_paths = []
labels = []

for emotion, paths in samples.items():
    audio_paths.extend(paths)
    labels.extend([emotion] * len(paths))

if not audio_paths:
    raise ValueError(
        "No suitable RAVDESS speech files found. "
        "Check the extracted dataset folder."
    )

print("Total selected audio files:", len(audio_paths))
print("Samples per emotion:", {
    emotion: len(paths) for emotion, paths in samples.items()
})

# 4. AUDIO PREPROCESSING AND MEL SPECTROGRAM EXTRACTION
SAMPLE_RATE = 22050
DURATION = 3
SAMPLES_PER_TRACK = SAMPLE_RATE * DURATION
N_MELS = 64
MAX_FRAMES = 128

X = []
y = []

for i, file_path in enumerate(audio_paths):
    try:
        audio, sr = librosa.load(
            file_path,
            sr=SAMPLE_RATE,

```

```

        duration=DURATION
    )

    # Pad or truncate audio to 3 seconds
    if len(audio) < SAMPLES_PER_TRACK:
        audio = np.pad(
            audio,
            (0, SAMPLES_PER_TRACK - len(audio))
        )
    else:
        audio = audio[:SAMPLES_PER_TRACK]

    # Extract Mel spectrogram
    mel = librosa.feature.melspectrogram(
        y=audio,
        sr=sr,
        n_fft=1024,
        hop_length=512,
        n_mels=N_MELS
    )

    mel_db = librosa.power_to_db(mel, ref=np.max)

    # Standardize the number of time frames
    mel_db = mel_db[:, :MAX_FRAMES]

    if mel_db.shape[1] < MAX_FRAMES:
        mel_db = np.pad(
            mel_db,
            (
                (0, 0),
                (0, MAX_FRAMES - mel_db.shape[1])
            )
        )

    X.append(mel_db)
    y.append(labels[i])

except Exception as e:
    print("Skipping file:", file_path, "|", e)

if (i + 1) % 100 == 0:
    print(f"Processed {i + 1}/{len(audio_paths)} files")

X = np.array(X, dtype=np.float32)
X = X[..., np.newaxis]

y = np.array(y)

print("Feature shape:", X.shape)
print("Label shape:", y.shape)

# 5. ENCODE LABELS AND SPLIT DATASET
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
y_categorical = to_categorical(y_encoded)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_categorical,
    test_size=0.20,
    random_state=42,
    stratify=y_encoded
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
print("Emotion classes:", list(label_encoder.classes_))

```

```

# 6. BUILD CNN MODEL
model = Sequential([
    tf.keras.Input(shape=(N_MELS, MAX_FRAMES, 1)),

    Conv2D(32, (3, 3), activation="relu", padding="same"),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), activation="relu", padding="same"),
    MaxPooling2D((2, 2)),

    Conv2D(128, (3, 3), activation="relu", padding="same"),
    GlobalAveragePooling2D(),

    Dropout(0.3),
    Dense(64, activation="relu"),
    Dropout(0.3),

    Dense(len(label_encoder.classes_), activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# 7. TRAIN CNN MODEL
history = model.fit(
    X_train,
    y_train,
    epochs=8,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

# 8. EVALUATE MODEL
test_loss, test_accuracy = model.evaluate(
    X_test, y_test, verbose=0
)

y_true = np.argmax(y_test, axis=1)
y_pred = np.argmax(model.predict(X_test, verbose=0), axis=1)

print("\nTest Accuracy:", round(test_accuracy * 100, 2), "%")
print("Test Loss:", round(test_loss, 4))

print("\nClassification Report:\n")
print(
    classification_report(
        y_true,
        y_pred,
        labels=np.arange(len(label_encoder.classes_)),
        target_names=label_encoder.classes_,
        zero_division=0
    )
)

# 9. CONFUSION MATRIX
cm = confusion_matrix(
    y_true,
    y_pred,
    labels=np.arange(len(label_encoder.classes_))
)

plt.figure(figsize=(9, 7))
sns.heatmap(
    cm,

```

```

    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)
plt.xlabel("Predicted Emotion")
plt.ylabel("Actual Emotion")
plt.title("CNN Speech Emotion Recognition - Confusion Matrix")
plt.tight_layout()
plt.show()

```

```

# 10. PLOT TRAINING AND VALIDATION ACCURACY
plt.figure(figsize=(8, 5))
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Training and Validation Accuracy")
plt.legend()
plt.grid(True)
plt.show()

```

```

# 11. SAVE TRAINED CNN MODEL
model.save("/content/speech_emotion_cnn.keras")

```

```

np.save(
    "/content/emotion_classes.npy",
    label_encoder.classes_
)

```

```

print("CNN model and emotion classes saved successfully.")

```

```

# 12. LOAD WHISPER SPEECH-TO-TEXT MODEL
device = 0 if torch.cuda.is_available() else -1

```

```

whisper = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-base",
    device=device
)

```

```

print("Whisper STT model loaded successfully.")

```

```

# 13. UPLOAD AUDIO FOR TESTING
print("Upload a WAV audio file to test the pipeline.")

```

```

uploaded_audio = files.upload()

```

```

audio_paths = [
    name for name in uploaded_audio
    if name.lower().endswith(".wav")
]

```

```

if not audio_paths:
    raise ValueError("Please upload a WAV audio file.")

```

```

test_audio_path = audio_paths[0]

```

```

print("Selected audio:", test_audio_path)
display(Audio(test_audio_path))

```

```

# 14. SPEECH-TO-TEXT TRANSCRIPTION
stt_result = whisper(test_audio_path)
transcription = stt_result["text"].strip()

```

```

print("Recognized Speech:", transcription)

```

```

# 15. PREDICT EMOTION FROM UPLOADED AUDIO
audio, sr = librosa.load(
    test_audio_path,
    sr=SAMPLE_RATE,
    duration=DURATION
)

if len(audio) < SAMPLES_PER_TRACK:
    audio = np.pad(
        audio,
        (0, SAMPLES_PER_TRACK - len(audio))
    )
else:
    audio = audio[:SAMPLES_PER_TRACK]

mel = librosa.feature.melspectrogram(
    y=audio,
    sr=sr,
    n_fft=1024,
    hop_length=512,
    n_mels=N_MELS
)

mel_db = librosa.power_to_db(mel, ref=np.max)
mel_db = mel_db[:, :MAX_FRAMES]

if mel_db.shape[1] < MAX_FRAMES:
    mel_db = np.pad(
        mel_db,
        (
            (0, 0),
            (0, MAX_FRAMES - mel_db.shape[1])
        )
    )

features = mel_db[np.newaxis, ..., np.newaxis].astype(np.float32)

probabilities = model.predict(features, verbose=0)[0]
predicted_index = np.argmax(probabilities)

predicted_emotion = label_encoder.inverse_transform(
    [predicted_index]
)[0]

confidence = probabilities[predicted_index] * 100

print("\nPredicted Emotion:", predicted_emotion.capitalize())
print(f"Confidence: {confidence:.2f}%")

# 16. TEXT-TO-SPEECH USING gTTS
if not transcription:
    print("No speech was recognized; skipping text-to-speech.")
else:
    tts = gTTS(text=transcription, lang="en")
    tts.save("/content/generated_speech.mp3")

    print("\nGenerated Speech:", transcription)
    display(Audio("/content/generated_speech.mp3", autoplay=False))

# 17. PIPELINE COMPLETE
print("\nSpeech Emotion Recognition Pipeline Completed!")
print("Steps: Audio Input -> CNN Emotion Prediction")
print("        -> Whisper Speech-to-Text -> gTTS Text-to-Speech")

```

## Output:

### 1] RAVDESS Dataset Upload and Emotion-Based Sample Selection.

```
uploaded = files.upload()

import zipfile

zip_files = [name for name in uploaded if name.lower().endswith(".zip")]

if zip_files:
    with zipfile.ZipFile(zip_files[0], "r") as zip_ref:
        zip_ref.extractall("/content/RAVDESS")

dataset_path = "/content/RAVDESS"

print("Dataset folder:", dataset_path)
print("Extraction completed.")
```

Created File: archive (3).zip  
archive (3).zip(application/x-zip-compressed) - 449906431 bytes, last modified: 10/10/2026 - 100% done  
Saving archive (3).zip to archive (3) (2).zip  
Dataset folder: /content/RAVDESS  
Extraction completed.

```
*** Speech files found: 1535
angry: selected 60 files
calm: selected 60 files
disgust: selected 60 files
fearful: selected 60 files
happy: selected 60 files
neutral: selected 60 files
sad: selected 60 files
surprised: selected 60 files

Total selected files: 480
```

### 2] Mel Spectrogram Feature Extraction and CNN Architecture.

```
*** Model: "sequential"
```

| Layer (type)                                      | Output Shape        | Param # |
|---------------------------------------------------|---------------------|---------|
| conv2d (Conv2D)                                   | (None, 64, 128, 32) | 320     |
| max_pooling2d (MaxPooling2D)                      | (None, 32, 64, 32)  | 0       |
| conv2d_1 (Conv2D)                                 | (None, 32, 64, 64)  | 18,496  |
| max_pooling2d_1 (MaxPooling2D)                    | (None, 16, 32, 64)  | 0       |
| conv2d_2 (Conv2D)                                 | (None, 16, 32, 128) | 73,856  |
| global_average_pooling2d (GlobalAveragePooling2D) | (None, 128)         | 0       |
| dropout (Dropout)                                 | (None, 128)         | 0       |
| dense (Dense)                                     | (None, 64)          | 8,256   |
| dense_1 (Dense)                                   | (None, 8)           | 520     |

Total params: 101,448 (396.28 KB)  
Trainable params: 101,448 (396.28 KB)  
Non-trainable params: 0 (0.00 B)

```
*** Processed 50/480 files
Processed 100/480 files
Processed 150/480 files
Processed 200/480 files
Processed 250/480 files
Processed 300/480 files
Processed 350/480 files
Processed 400/480 files
Processed 450/480 files

Feature extraction completed.
X shape: (480, 64, 128, 1)
Labels shape: (480,)
```

### 3] CNN Model Training and Validation.

```
history = model.fit(
    X_train,
    y_train,
    epochs=8,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)
```

```
*** Epoch 1/8
10/10 ----- 14s 736ms/step - accuracy: 0.1205 - loss: 3.9327 - val_accuracy: 0.1169 - val_loss: 2.2256
Epoch 2/8
10/10 ----- 0s 23ms/step - accuracy: 0.1336 - loss: 2.1765 - val_accuracy: 0.0779 - val_loss: 2.1031
Epoch 3/8
10/10 ----- 0s 21ms/step - accuracy: 0.1368 - loss: 2.0790 - val_accuracy: 0.1169 - val_loss: 2.0766
Epoch 4/8
10/10 ----- 0s 28ms/step - accuracy: 0.1205 - loss: 2.0797 - val_accuracy: 0.1169 - val_loss: 2.0800
Epoch 5/8
10/10 ----- 0s 24ms/step - accuracy: 0.1433 - loss: 2.0786 - val_accuracy: 0.1429 - val_loss: 2.0791
Epoch 6/8
10/10 ----- 0s 23ms/step - accuracy: 0.1596 - loss: 2.0774 - val_accuracy: 0.1688 - val_loss: 2.0790
Epoch 7/8
10/10 ----- 0s 27ms/step - accuracy: 0.1498 - loss: 2.0787 - val_accuracy: 0.2078 - val_loss: 2.0798
Epoch 8/8
10/10 ----- 0s 26ms/step - accuracy: 0.1238 - loss: 2.0762 - val_accuracy: 0.1429 - val_loss: 2.0797
```

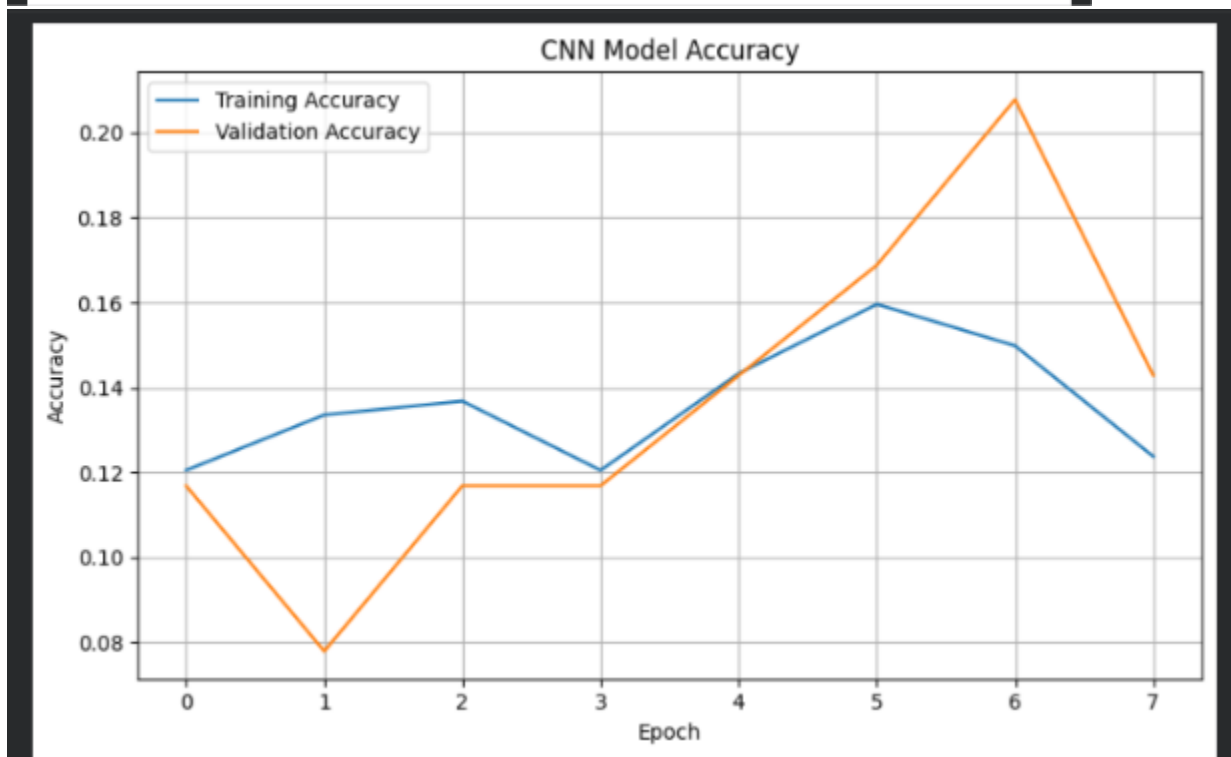
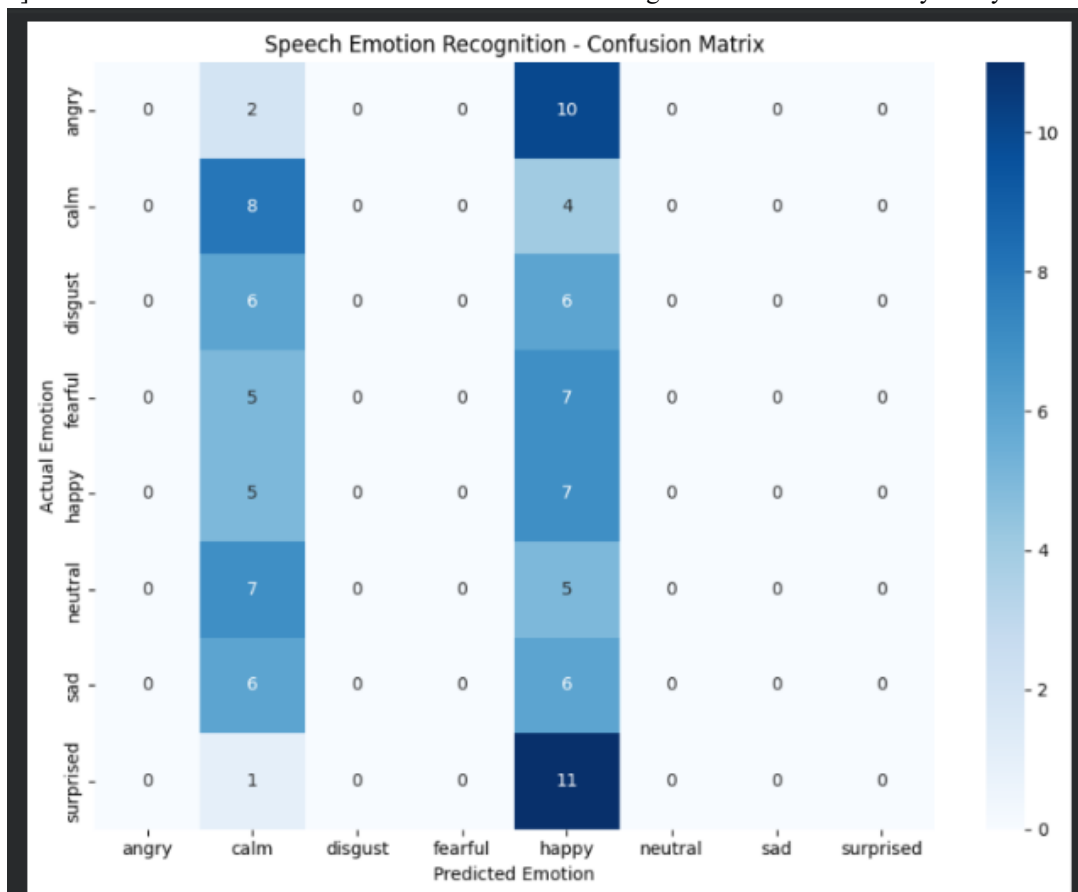
### 4] CNN Model Evaluation and Classification Report.

```
*** Test Loss: 2.0763
Test Accuracy: 0.1562
Test Accuracy Percentage: 15.62%

Classification Report:
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| angry        | 0.00      | 0.00   | 0.00     | 12      |
| calm         | 0.20      | 0.67   | 0.31     | 12      |
| disgust      | 0.00      | 0.00   | 0.00     | 12      |
| fearful      | 0.00      | 0.00   | 0.00     | 12      |
| happy        | 0.12      | 0.58   | 0.21     | 12      |
| neutral      | 0.00      | 0.00   | 0.00     | 12      |
| sad          | 0.00      | 0.00   | 0.00     | 12      |
| surprised    | 0.00      | 0.00   | 0.00     | 12      |
| accuracy     |           |        | 0.16     | 96      |
| macro avg    | 0.04      | 0.16   | 0.06     | 96      |
| weighted avg | 0.04      | 0.16   | 0.06     | 96      |

5] Confusion Matrix for Emotion Classification and Training and Validation Accuracy Analysis.



6] Test Audio Upload and Selection.

```
from google.colab import files
from IPython.display import Audio, display
import os

uploaded_audio = files.upload()

audio_paths = [
    name for name in uploaded_audio
    if name.lower().endswith(".wav")
]

if not audio_paths:
    raise ValueError("Please upload a WAV audio file.")

test_audio_path = audio_paths[0]

print("Selected audio:", test_audio_path)
```

Choose Files file\_examp...V\_1MG.wav  
file\_example\_WAV\_1MG.wav(audio/wav) - 1048558 bytes, last modified: 10/10/2026 - 100% done  
Saving file\_example\_WAV\_1MG.wav to file\_example\_WAV\_1MG.wav  
Selected audio: file\_example\_WAV\_1MG.wav

7] Speech Emotion Prediction and Audio Playback.

```
*** Predicted Emotion: Calm
    Confidence: 13.08%
```



```
*** Generated Speech: Thanks for watching!
```

